

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\workflows\wf_095035a0-e9f\agent-a9694783d8dcb8a0a.jsonl`  
- SHA-256: `758f2ff57864952376a03ee3cc03614d19decdb695bd83ab0b441ade142f36d3`  
- Source modified: `2026-06-13T19:13:01+00:00`  
- Imported at: `2026-07-08T16:00:00+00:00`  
- project: `wf_095035a0-e9f`  
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`
```

Transcript

1. user (2026-06-13T19:09:22.950Z)

Audit the tests (tests/test_fusion_features.py, test_fusion_hybrid.py, test_trajectory_hybrid_equivalence.py additions, test_config.py additions). What real behaviours are UNtested that COULD be tested in a no-GPU container? Any test that asserts the wrong thing or passes vacuously? Distinguish genuinely-GPU-only gaps from gaps that should be covered here. Be specific.

Repo root = C:/Users/avidu/Projects/badminton-highlight-indexer. The change implements fusion P2 (MULTI_SIGNAL_FUSION_PLAN.md): a new default-OFF `FusionSegmenter` (backend/pipeline/segmenters/fusion_hybrid.py) that subclasses TrajectoryHybridSegmenter, persists shuttle net_crossings (Gate A) + optional person-cue features into candidate stats, and optionally gates the AI handoff. Pure feature math is in backend/pipeline/detectors/fusion_features.py. NEW files: backend/pipeline/detectors/fusion_features.py, backend/pipeline/segmenters/fusion_hybrid.py. EDITED files (use `git --no-pager diff -- <path>` to see edits): backend/pipeline/segmenters/trajectory_hybrid.py, backend/pipeline/detectors/person_detector.py, backend/config/models.py, backend/pipeline/segmenters/__init__.py. READ the actual files. The work is uncommitted in the working tree on branch master. Quote file:line. Only report REAL issues; rank by severity (high/med/low).

2. assistant (2026-06-13T19:09:26.386Z)

I'll audit the fusion P2 test additions. Let me start by reading the actual implementation and test files.

3. user (2026-06-13T19:09:29.330Z)

```
fatal: Cannot fast-forward to multiple branches.  
---STATUS---  
M backend/config/models.py  
M backend/pipeline/detectors/person_detector.py  
M backend/pipeline/segmenters/__init__.py  
M backend/pipeline/segmenters/trajectory_hybrid.py  
M docs/README.md  
M tests/test_config.py  
M tests/test_trajectory_hybrid_equivalence.py  
?? artifacts/  
?? backend/pipeline/detectors/fusion_features.py  
?? backend/pipeline/segmenters/fusion_hybrid.py  
?? docs/QUALITY_ITERATIONS.md  
?? scratch/HIGH_RISK_REMEDIATION_PLAN.md  
?? scratch/PR1_BODY.md  
?? scratch/PR1_HANDOFF.md  
?? scratch/PR2_HANDOFF.md  
?? scratch/PR2_deferred_hardening_status.patch
```

```

?? scratch/PR88_FIX.md
?? scratch/arial.ttf
?? scratch/debug_wsl.py
?? scratch/keep_awake.ps1
?? scratch/probe_cv2_count.py
?? scratch/probe_gemini.py
?? scratch/repro_drawtext_font.py
?? scratch/repro_movie_overlay.py
?? scratch/run_gx010125.py
?? scratch/stitch_min5.py
?? scratch/trim_video.py
?? tests/test_fusion_features.py
?? tests/test_fusion_hybrid.py

```

4. assistant (2026-06-13T19:09:32.399Z)

Note: the `git pull --ff-only` failed (multiple branches tracking issue), so I'm working read-only on the local tree, which matches the prompt's statement that this work is uncommitted. Let me read all the relevant files.

5. user (2026-06-13T19:09:33.355Z)

```

1      """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).
2
3      NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4      math is unit-testable in a GPU-free dev container. The detection (person boxes,
5      shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6      module only turns those signals into a handful of scale/translation-invariant
7      features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8      memorise this court/camera and fail to generalise off a small labelled set).
9
10     Conventions:
11     - A person box is `(x1, y1, x2, y2)` in pixels; `(track_id, box)` when motion
needs it.
12     - `per_frame` is `{frame_idx: [(track_id, box), ...]}` keyed by the ORIGINAL video
frame
13     index (so it aligns directly with the shuttle trajectory's frame ? no clip re-
timing).
14     - A shuttle point has frame, visible, x, y (pixels) ? the WASB
trajectory shape.
15     - court_polygon is normalised 0-1 [(x, y), ...] or None (None ? no mask,
every
16     detection counts ? the player-count-only mode).
17     - net is (axis 'x'|'y', position 0-1 normalised) or None (None ? two-sided =
0.0).
18
19     Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20     floats in [0, 1] except mean_player_motion (a non-negative normalised speed).
21     """
22     from __future__ import annotations
23
24     from typing import Dict, List, Optional, Sequence, Tuple
25
26     BBox = Tuple[float, float, float, float]
27     Net = Tuple[str, float]
28
29
30     def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
31         """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32         (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
33         if polygon is None or len(polygon) < 3:
34             return False
35         x, y = point

```

```

36         inside = False
37         n = len(polygon)
38         j = n - 1
39         for i in range(n):
40             xi, yi = polygon[i]
41             xj, yj = polygon[j]
42             # Does the horizontal ray at y cross edge (i, j)?
43             if (yi > y) != (yj > y):
44                 x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
45                 if x < x_cross:
46                     inside = not inside
47             j = i
48         return inside
49
50
51     def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
52         """Normalised foot point (bottom-centre) of a person box ? where the player stands,
53         the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
54         if frame_width <= 0 or frame_height <= 0:
55             return (0.0, 0.0)
56         x1, _y1, x2, y2 = box
57         return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)
58
59
60     def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
61         if frame_width <= 0 or frame_height <= 0:
62             return (0.0, 0.0)
63         x1, y1, x2, y2 = box
64         return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)
65
66
67     def person_in_court(box: BBox, frame_width: float, frame_height: float,
68                        court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
69         """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
70         True (count everyone) when it is None."""
71         if court_polygon is None:
72             return True
73         return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)
74
75
76     def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
77                        frame_height: float,
78                        court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:
79         """Per-frame count of in-court persons (one entry per sampled frame)."""
80         return [
81             sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
82             court_polygon))
83             for _frame_idx, dets in sorted(per_frame.items())
84         ]
85
86     def _median(vals: Sequence[float]) -> float:
87         if not vals:
88             return 0.0
89         s = sorted(vals)
90         n = len(s)
91         return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))
92
93
94     def players_in_court(counts: Sequence[int]) -> float:
95         """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
96         return _median(list(counts))

```

```

97
98
99     def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
100         """Fraction of sampled frames with >= `min_players` in court (track stability vs
flicker)."""
101         if not counts:
102             return 0.0
103         return sum(1 for c in counts if c >= min_players) / len(counts)
104
105
106     def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
107                             frame_width: float, frame_height: float) -> float:
108         """Mean per-track centre displacement between consecutive sampled frames, normalised
109         by the frame width (resolution-invariant). 0.0 if <2 frames or no shared tracks."""
110         if frame_width <= 0 or len(per_frame) < 2:
111             return 0.0
112         frames = sorted(per_frame.keys())
113         steps: List[float] = []
114         for a, b in zip(frames, frames[1:]):
115             prev = {tid: box for tid, box in per_frame[a]}
116             for tid, box in per_frame[b]:
117                 if tid in prev:
118                     cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
119                     cx1, cy1 = _center_norm(box, frame_width, frame_height)
120                     steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
121         if not steps:
122             return 0.0
123         return sum(steps) / len(steps)
124
125
126     def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
127                             frame_height: float, net: Optional[Net],
128                             court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
129         """Fraction of sampled frames with >=1 in-court person on BOTH net halves.
130
131         A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
132         The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
0.0.
133
134         """
135         if net is None or not per_frame:
136             return 0.0
137         axis, pos = net
138         both = 0
139         for _frame_idx, dets in per_frame.items():
140             near, far = False, False
141             for _tid, box in dets:
142                 if not person_in_court(box, frame_width, frame_height, court_polygon):
143                     continue
144                 fx, fy = _foot_norm(box, frame_width, frame_height)
145                 coord = fx if axis == "x" else fy
146                 if coord < pos:
147                     near = True
148                 else:
149                     far = True
150             if near and far:
151                 both += 1
152         return both / len(per_frame)
153
154     def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
float,
155                             pad_frac: float) -> bool:
156         """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
157         width/height (the 'adjacent / in-hand' tolerance)?"
158         x1, y1, x2, y2 = box

```

```

159     px = pad_frac * frame_width
160     py = pad_frac * frame_height
161     return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)
162
163
164     def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
165                                     frame_width: float, frame_height: float,
166                                     pad_frac: float = 0.03) -> float:
167         """Fraction of (visible shuttle frames that were also person-sampled) in which the
168         shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
~0**
169         ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
170         """
171         if not per_frame or frame_width <= 0 or frame_height <= 0:
172             return 0.0
173         aligned = 0
174         colocated = 0
175         for p in shuttle_points or []:
176             if not getattr(p, "visible", False):
177                 continue
178             dets = per_frame.get(int(p.frame))
179             if dets is None:         # shuttle frame wasn't person-sampled ? can't judge
180                 continue
181             aligned += 1
182             if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
183                   for _tid, box in dets):
184                 colocated += 1
185         return colocated / aligned if aligned else 0.0
186
187
188     # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
189     # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
190     PERSON_FEATURE_NAMES = [
191         "players_in_court", "two_sided_motion", "mean_player_motion",
192         "in_court_ratio", "shuttle_player_colocation",
193     ]
194
195
196     def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
197                                     frame_width: float, frame_height: float,
198                                     court_polygon: Optional[Sequence[Tuple[float, float]]] =
None,
199                                     net: Optional[Net] = None,
200                                     min_players: int = 2) -> Dict[str, float]:
201         """Aggregate the per-window person features from per-frame detections + the in-
window
202         shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
203         counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
204         return {
205             "players_in_court": players_in_court(counts),
206             "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
court_polygon),
207             "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
208             "in_court_ratio": in_court_ratio(counts, min_players),
209             "shuttle_player_colocation": shuttle_player_colocation(
210                 per_frame, shuttle_points, frame_width, frame_height),
211         }
212

```

6. user (2026-06-13T19:09:33.776Z)

```

1     """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2

```

```

3 Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4 TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5 do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6 shuttle-seeded window via two overridable seams the engine exposes:
7
8 - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9 GPU-free, computed from the trajectory we already have); and, only when the person cue
10 is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11 (``fusion_features``), persisting them into ``candidate_segments.stats`` for the learned
12 fusion (P3). The person path lazily builds ONE `PersonDetector` ? so the default path
13 never imports torch / touches the GPU.
14 - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
15 (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
16 dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
17 **Persisted candidates remain the full superset** regardless, so the offline
experiment
18 harness can re-score any variant.
19
20 Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
21 nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
22 and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
23 segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
24 experiment harness (EXPERIMENT_HARNESS.md), never silently.
25 """
26 from typing import Any, Dict, List, Optional, Tuple
27
28 from backend.pipeline.segmenters.base import segmenter_registry
29 from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
30 from backend.pipeline.detectors.base import DetectorRunner
31 from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
32 from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
33 from backend.pipeline.detectors import rally_gate
34 from backend.pipeline.detectors import fusion_features as ff
35
36
37 class FusionSegmenter(TrajectoryHybridSegmenter):
38     """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
B)."""
39
40     family = "fusion"
41     display_label = "Fusion"
42
43     def __init__(self, db: Any, config: Any):
44         super().__init__(db, config)
45         # Built lazily only if the person cue is enabled (keeps torch/GPU off the
default path).
46         self._person: Optional[Any] = None
47
48     @property
49     def name(self) -> str:
50         return "fusion_hybrid"
51
52     @property
53     def description(self) -> str:
54         return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
rallies, "
55                 "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
and the "
56                 "AI provider sets semantic boundaries.")

```



```

111             idx_cfg: Dict[str, Any]) -> List[int]:
112         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
113         (0) keep every window ? identical to the substrate. Persisted candidates are
114         untouched (full superset)."""
115         fcfg = idx_cfg.get("fusion", {})
116         min_crossings = int(fcfg.get("min_crossings", 0))
117         min_players = int(fcfg.get("min_players", 0))
118         person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
119
120         keep: List[int] = []
121         for i, st in enumerate(window_stats):
122             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
123                 continue
124             if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
125                 continue
126             keep.append(i)
127         return keep
128
129
130     segmenter_registry.register(FusionSegmenter)
131

```

7. assistant (2026-06-13T19:09:35.933Z)

Now let me read the test files and the edited implementation files.

8. user (2026-06-13T19:09:39.563Z)

```

1     """Pure-logic unit tests for the person-cue fusion features
(backend.pipeline.detectors.fusion_features).
2
3     No GPU/torch/cv2 ? synthetic person boxes + shuttle points exercise every feature path
and
4     edge case. This is the testable core that proves the fusion math correct before any GPU
run.
5     """
6     from types import SimpleNamespace
7
8     import pytest
9
10    from backend.pipeline.detectors import fusion_features as ff
11
12
13    def _shuttle(frame, x, y, visible=True):
14        return SimpleNamespace(frame=frame, x=x, y=y, visible=visible)
15
16
17    # Square unit polygon and a left-half polygon (x in [0, 0.5]).
18    FULL = [(0.0, 0.0), (1.0, 0.0), (1.0, 1.0), (0.0, 1.0)]
19    LEFT_HALF = [(0.0, 0.0), (0.5, 0.0), (0.5, 1.0), (0.0, 1.0)]
20
21
22    # ----- point_in_polygon
23
24    def test_point_in_polygon_inside_outside():
25        assert ff.point_in_polygon((0.5, 0.5), FULL) is True
26        assert ff.point_in_polygon((1.5, 0.5), FULL) is False
27        assert ff.point_in_polygon((0.2, 0.5), LEFT_HALF) is True
28        assert ff.point_in_polygon((0.8, 0.5), LEFT_HALF) is False
29
30
31    def test_point_in_polygon_degenerate():
32        assert ff.point_in_polygon((0.5, 0.5), None) is False

```



```

33         assert ff.point_in_polygon((0.5, 0.5), [(0.0, 0.0), (1.0, 1.0)]) is False # <3
verts
34
35
36     # ----- person_in_court
37
38     def test_person_in_court_uses_foot_point_and_polygon():
39         # box foot = ((x1+x2)/2/W, y2/H). W=H=100.
40         in_box = (10, 10, 30, 30) # foot (0.2, 0.3) ? left half
41         out_box = (60, 10, 80, 30) # foot (0.7, 0.3) ? right half
42         assert ff.person_in_court(in_box, 100, 100, LEFT_HALF) is True
43         assert ff.person_in_court(out_box, 100, 100, LEFT_HALF) is False
44         # None polygon => everyone counts
45         assert ff.person_in_court(out_box, 100, 100, None) is True
46         # Unusable frame dims => foot (0,0); with a polygon that excludes origin => False
47         assert ff.person_in_court(in_box, 0, 0, LEFT_HALF) is True # (0,0) is on left half
edge-inside
48
49
50     # ----- counts / ratios
51
52     def test_in_court_counts_and_players_and_ratio():
53         near = (10, 10, 30, 30) # foot (0.2,0.3) in LEFT_HALF
54         far = (60, 10, 80, 30) # foot (0.7,0.3) NOT in LEFT_HALF
55         per_frame = {0: [(1, near), (2, far)], 1: [(1, near)], 2: [], 3: [(1, near), (2,
near)]}]
56         # With LEFT_HALF mask: counts = [1, 1, 0, 2]
57         counts = ff.in_court_counts(per_frame, 100, 100, LEFT_HALF)
58         assert counts == [1, 1, 0, 2]
59         assert ff.players_in_court(counts) == 1.0 # median of [0,1,1,2]
60         assert ff.in_court_ratio(counts, min_players=2) == pytest.approx(0.25)
61         assert ff.in_court_ratio(counts, min_players=1) == pytest.approx(0.75)
62         # No mask: far counts too -> [2,1,0,2]
63         assert ff.in_court_counts(per_frame, 100, 100, None) == [2, 1, 0, 2]
64         assert ff.players_in_court([]) == 0.0
65         assert ff.in_court_ratio([]) == 0.0
66
67
68     # ----- mean_player_motion
69
70     def test_mean_player_motion():
71         # track 1 moves 10px in x between frames; W=H=100 -> normalized disp 0.10.
72         per_frame = {0: [(1, (0, 0, 10, 10))], 1: [(1, (10, 0, 20, 10))]}
73         assert ff.mean_player_motion(per_frame, 100, 100) == pytest.approx(0.10)
74         # single frame -> 0; no shared track id -> 0
75         assert ff.mean_player_motion({0: [(1, (0, 0, 10, 10))]}, 100, 100) == 0.0
76         assert ff.mean_player_motion({0: [(1, (0, 0, 10, 10))], 1: [(2, (10, 0, 20, 10))]},
100, 100) == 0.0
77
78
79     # ----- two_sided_motion
80
81     def test_two_sided_motion():
82         net = ("y", 0.5)
83         near = (10, 10, 30, 40) # foot y = 0.4 < 0.5 (near side)
84         far = (10, 60, 30, 90) # foot y = 0.9 >= 0.5 (far side)
85         both = {0: [(1, near), (2, far)], 1: [(1, near)]} # frame 0 two-sided, frame 1
one-sided
86         assert ff.two_sided_motion(both, 100, 100, net, None) == pytest.approx(0.5)
87         one_sided = {0: [(1, near)], 1: [(2, near)]}
88         assert ff.two_sided_motion(one_sided, 100, 100, net, None) == 0.0
89         assert ff.two_sided_motion(both, 100, 100, None, None) == 0.0 # no net -> 0
90
91
92     # ----- shuttle_player_colocation

```

```

93
94     def test_shuttle_player_colocation():
95         box = (40, 40, 60, 60)
96         per_frame = {0: [(1, box)], 1: [(1, box)], 2: [(1, box)]}
97         # frame0 shuttle inside box (held), frame1 shuttle far (in-flight), frame2
invisible.
98         shuttle = [_shuttle(0, 50, 50, True), _shuttle(1, 95, 95, True), _shuttle(2, 50, 50,
False)]
99         # aligned visible frames = {0, 1}; colocated = {0} -> 0.5
100         assert ff.shuttle_player_colocation(per_frame, shuttle, 100, 100) ==
pytest.approx(0.5)
101         # all-held -> 1.0
102         held = [_shuttle(0, 50, 50, True), _shuttle(1, 50, 50, True)]
103         assert ff.shuttle_player_colocation(per_frame, held, 100, 100) == pytest.approx(1.0)
104         # no shuttle -> 0.0; shuttle frame not person-sampled -> skipped (no aligned -> 0.0)
105         assert ff.shuttle_player_colocation(per_frame, [], 100, 100) == 0.0
106         assert ff.shuttle_player_colocation(per_frame, [_shuttle(99, 50, 50, True)], 100,
100) == 0.0
107
108
109     # ----- aggregator
110
111     def test_compute_person_features_keys():
112         near = (10, 10, 30, 40)
113         far = (10, 60, 30, 90)
114         per_frame = {0: [(1, near), (2, far)], 1: [(1, near), (2, far)]}
115         shuttle = [_shuttle(0, 20, 25, True)]
116         feats = ff.compute_person_features(per_frame, shuttle, 100, 100,
117                                           court_polygon=None, net="y", 0.5))
118         assert set(feats) == set(ff.PERSON_FEATURE_NAMES)
119         assert all(isinstance(v, float) for v in feats.values())
120         assert feats["two_sided_motion"] == pytest.approx(1.0) # both sides occupied every
frame
121
122
123     def test_compute_person_features_empty():
124         feats = ff.compute_person_features({}, [], 100, 100)
125         assert set(feats) == set(ff.PERSON_FEATURE_NAMES)
126         assert all(v == 0.0 for v in feats.values())
127

```

9. user (2026-06-13T19:09:39.565Z)

```

1     """FusionSegmenter tests (mocked runner/provider/db ? no GPU/WSL).
2
3     Proves: net_crossings is persisted; the person cue is OFF by default (no person-feature
4     keys, PersonDetector never built); the persisted candidate set is the full superset of
the
5     substrate windows; served Gate A drops sub-threshold windows from the AI handoff while
6     persistence keeps the superset; and the person cue, when enabled, adds the person
features.
7     """
8     from types import SimpleNamespace
9     from unittest.mock import MagicMock, patch
10
11     from backend.pipeline.detectors.fusion_features import PERSON_FEATURE_NAMES
12     from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
13
14     # Window A [1,4]s: shuttle alternates across the net every frame -> many crossings (a
rally).
15     # Window B [10,12]s: shuttle sits at the net level (held) -> 0 crossings. fps=30.
16     WIN_A = {"start": 1.0, "end": 4.0, "active_frame_count": 90, "peak_velocity": 0.4,
17             "mean_velocity": 0.2, "track_id_count": 2, "chunk_index": 0}
18     WIN_B = {"start": 10.0, "end": 12.0, "active_frame_count": 60, "peak_velocity": 0.1,
19             "mean_velocity": 0.05, "track_id_count": 1, "chunk_index": 0}

```

```

20
21
22     def _points():
23         pts = []
24         for f in range(30, 120):             # window A frames: y flips 100<->900 around the net
25             pts.append(SimpleNamespace(frame=f, visible=True, x=500.0, y=100.0 if f % 2 == 0
26             else 900.0))
27         for f in range(300, 360):             # window B frames: y == net level -> deadband -> no
28             pts.append(SimpleNamespace(frame=f, visible=True, x=500.0, y=500.0))
29         return pts
30
31     def _runner():
32         r = MagicMock()
33         r.healthcheck.return_value = (True, "env ok")
34         r.run_predict.return_value = "out.csv"
35         r.parse_trajectory_csv.return_value = _points()
36         r.trajectory_to_action_windows.return_value = [dict(WIN_A), dict(WIN_B)]
37         return r
38
39
40     def _run(indexing=None, provider_segments=None):
41         provider = MagicMock()
42         provider.analyze_video.return_value = (provider_segments or [{"start_time": 0.5,
43         "end_time": 2.0}], {})
44         db = MagicMock()
45         db.add_candidate_segment.side_effect = [101, 102, 103, 104]
46         db.add_play_segment.return_value = 200
47
48         with patch.object(FusionSegmenter, "_build_runner", return_value=_runner(),
49         {"weights_path": "w"})), \
50             patch.object(FusionSegmenter, "_probe_fps_width", return_value=(30.0, 1280.0)), \
51             patch("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
52             return_value=30.0), \
53             patch("backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk",
54             return_value=True), \
55             patch("backend.pipeline.segmenters.trajectory_hybrid.provider_registry") as
56             preg, \
57             patch("os.environ.get", return_value="dummy_key"):
58             preg.get.return_value = provider
59             seg = FusionSegmenter(db, {"indexing": indexing or {}})
60             res = seg.process_video("v1", "clip.mp4")
61             return db, provider, res
62
63
64     def _stats_calls(db):
65         """Persisted stats dicts, in call order."""
66         return [kw["stats"] for _a, kw in db.add_candidate_segment.call_args_list]
67
68
69     def test_net_crossings_persisted_and_person_off_by_default():
70         db, _provider, _res = _run()
71         stats = _stats_calls(db)
72         assert len(stats) == 2                                     # both windows persisted
73         (superset)
74         assert all("net_crossings" in s for s in stats)
75         # Window A (rally) crosses many times; window B (held) zero.
76         assert stats[0]["net_crossings"] >= 2
77         assert stats[1]["net_crossings"] == 0.0
78         # Person cue OFF by default -> NONE of the person features present.
79         assert all(not any(k in s for k in PERSON_FEATURE_NAMES) for s in stats)
80         # The 4 base motion keys are still there.

```

```

75         assert all({"peak_velocity", "mean_velocity", "active_frame_count",
"track_id_count"} <= set(s)
76             for s in stats)
77
78
79     def test_persisted_candidates_are_superset_of_substrate_windows():
80         db, _provider, _res = _run()
81         spans = [(kw["start_time"], kw["end_time"]) for _a, kw in
db.add_candidate_segment.call_args_list]
82         assert spans == [(1.0, 4.0), (10.0, 12.0)]           # exactly the substrate windows,
unfiltered
83
84
85     def test_default_hands_off_all_windows():
86         _db, provider, _res = _run()                         # min_crossings default 0 -> no
gating
87         assert provider.analyze_video.call_count == 2
88
89
90     def test_gate_a_drops_low_crossing_window_from_handoff_only():
91         db, provider, _res = _run(indexing={"fusion": {"min_crossings": 2}})
92         # Persistence is still the full superset (offline substrate intact)...
93         assert len(_stats_calls(db)) == 2
94         # ...but only the high-crossing window A reaches the AI handoff.
95         assert provider.analyze_video.call_count == 1
96
97
98     def test_person_cue_adds_features_when_enabled():
99         fake_pd = MagicMock()
100         fake_pd.detections_per_frame.return_value = {
101             "frames": {30: [(1, (10, 10, 30, 40))], 31: [(1, (12, 10, 32, 40))]},
102             "fps": 30.0, "frame_width": 1000.0, "frame_height": 1000.0,
103         }
104         with patch("backend.pipeline.detectors.person_detector.PersonDetector",
return_value=fake_pd):
105             db, _provider, _res = _run(indexing={"fusion": {"cue_flags": {"person": True}}})
106             stats = _stats_calls(db)
107             assert all(all(k in s for k in PERSON_FEATURE_NAMES) for s in stats)
108             assert all("net_crossings" in s for s in stats)
109             fake_pd.detections_per_frame.assert_called()       # the person detector WAS used
110

```

10. assistant (2026-06-13T19:09:43.257Z)

Now let me read the edited implementation files to understand the seams and what's actually exercised.

11. user (2026-06-13T19:09:45.050Z)

warning: in the working copy of 'backend/pipeline/segmenters/trajectory_hybrid.py', LF will be replaced by CRLF the next time Git touches it

```

diff --git a/backend/pipeline/segmenters/trajectory_hybrid.py
b/backend/pipeline/segmenters/trajectory_hybrid.py
index b2847c9..149282e 100644

```

```

--- a/backend/pipeline/segmenters/trajectory_hybrid.py
+++ b/backend/pipeline/segmenters/trajectory_hybrid.py
@@ -78,6 +78,31 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
     except (ValueError, TypeError):
         return max(3, int(duration / 0.8))

```

```

+ # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged) -----
+ def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
+                             frame_width: float, video_path: str,
+                             idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
+     """Stats dict persisted into ``candidate_segments.stats`` for one window. The BASE

```

```

+         returns exactly the 4 motion keys, so the trajectory twins persist byte-identical
+         rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
+         features). ``points`` are the whole-video trajectory points (TrajectoryPoint)."""
+         return {
+             "peak_velocity": cw["peak_velocity"],
+             "mean_velocity": cw["mean_velocity"],
+             "active_frame_count": cw["active_frame_count"],
+             "track_id_count": cw["track_id_count"],
+         }
+
+     def _select_for_handoff(self, windows: List[Dict[str, Any]],
+                             window_stats: List[Dict[str, Any]],
+                             idx_cfg: Dict[str, Any]) -> List[int]:
+         """Indices of the candidate windows that proceed to the AI handoff (and thus may
+         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
+         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
+         Persisted candidates are NOT affected ? they remain the full superset for offline
+         re-scoring."""
+         return list(range(len(windows)))
+
+     def process_video(self, video_id: str, video_path: str, # type: ignore[override]
+                       player_pool: Optional[List[str]] = None,
+                       max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
+@@ -172,31 +197,36 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):
+         f"({cw['end'] - cw['start']:.1f}s) |
+ frames={cw['active_frame_count']} "
+         f"peak_v={cw['peak_velocity']:.3f}
+ mean_v={cw['mean_velocity']:.3f}")
+
+     # Per-window stats via the enrichment hook ? base = the 4 motion keys; the fusion
+     # segmenter adds net_crossings + person-cue features. Computed once, reused for both
+     # persistence and the handoff gate below.
+     window_stats = [
+         self._enrich_candidate_stats(cw, points, fps, frame_width, video_path, idx_cfg)
+         for cw in all_candidate_windows
+     ]
+
+     # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
+     candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
+     if store_candidates:
+         for i, cw in enumerate(all_candidate_windows):
+             stats = {
+                 "peak_velocity": cw["peak_velocity"],
+                 "mean_velocity": cw["mean_velocity"],
+                 "active_frame_count": cw["active_frame_count"],
+                 "track_id_count": cw["track_id_count"],
+             }
+             candidate_ids[i] = self.db.add_candidate_segment(
+                 video_id, detector=self.name,
+                 start_time=cw["start"], end_time=cw["end"],
+                 chunk_index=cw["chunk_index"], confidence=min(1.0, cw["peak_velocity"]),
+                 stats=stats, detection_params=detection_params,
+                 stats=window_stats[i], detection_params=detection_params,
+             )
+
+     if not all_candidate_windows:
+         return []
+
+     # --- Phase 3: AI provider handoff over padded candidate clips ---
+     # Which windows reach the handoff (base = all; fusion may apply Gate A/B). Persisted
+     # candidates above stay the full superset ? only the served play_segments are gated.
+     handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
+ idx_cfg)
+     handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
+     os.makedirs(handoff_dir, exist_ok=True)

```

```

        logger.info("Phase 3: %s validation on %d candidate windows...",
-            provider_name, len(all_candidate_windows))
+            provider_name, len(handoff_indices))

    def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
        w_start = max(0, window["start"] - handoff_padding)
@@ -225,8 +255,8 @@ class TrajectoryHybridSegmenter(BasePlaySegmenter):

        ai_segments: List[dict] = []
        with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as executor:
-            futures = [executor.submit(process_candidate_window, wi, cw)
-                for wi, cw in enumerate(all_candidate_windows)]
+            futures = [executor.submit(process_candidate_window, wi, all_candidate_windows[wi])
+                for wi in handoff_indices]
        for future in futures:
            ai_segments.extend(future.result())
    try:

```

12. user (2026-06-13T19:09:46.043Z)

```
diff --git a/backend/config/models.py b/backend/config/models.py
```

```
index bb56881..7f8191c 100644
```

```
--- a/backend/config/models.py
```

```
+++ b/backend/config/models.py
```

```
@@ -75,6 +75,39 @@ class WasbIndexConfig(BaseModel):
```

```
    min_free_gb: float = 0.0
```

```
+class FusionConfig(BaseModel):
```

```
+    """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
```

```
+    """
```

```
+    Every default reproduces control behaviour: the fusion segmenter persists the
+    shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']`` is
+    true ? the person-cue features, but it does NOT gate the served handoff unless a
+    threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court mask
+    is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-only);
+    supplied ? off-court persons (spectators / adjacent courts) are excluded.
```

```
+    """
```

```
+    # Which trajectory substrate seeds the windows (shuttle stays primary).
```

```
+    substrate: Literal["wasb", "tracknet"] = "wasb"
```

```
+    # Windowing velocity threshold for the fusion family (the engine reads
```

```
+    # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
```

```
+    # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
```

```
+    velocity_threshold: float = 0.02
```

```
+    # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
```

```
+    # enabled ? so the default path never imports torch / touches the GPU.
```

```
+    cue_flags: dict[str, bool] = Field(default_factory=dict)
```

```
+    # Served Gate A: drop windows whose shuttle net-crossings < this from the AI handoff.
```

```
+    # 0 = OFF (no gating; persisted candidates are always the full superset for offline re-
+    scoring).
```

```
+    min_crossings: int = Field(default=0, ge=0)
```

```
+    # Served Gate B: when the person cue is on, drop windows with median in-court players
```

```
+    # < this. 0 = OFF.
```

```
+    min_players: int = Field(default=0, ge=0)
```

```
+    # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
```

```
+    court_polygon: Optional[list[list[float]]] = None
```

```
+    # Net line for the person two-sided-motion split (person features only ? the shuttle
```

```
+    # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
```

```
+    net_axis: Literal["x", "y"] = "y"
```

```
+    net_position: float = Field(default=0.5, ge=0.0, le=1.0)
```

```
+    """
```

```
+    """
```

```
class IndexingConfig(BaseModel):
```

```
    default_segmenter: str = "yolo_hybrid"
```

```
    use_gpu: bool = True
```

```

@@ -106,6 +139,7 @@ class IndexingConfig(BaseModel):

    tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
    wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
+    fusion: FusionConfig = Field(default_factory=FusionConfig)

    @field_validator("default_segmenter")
    @classmethod
diff --git a/backend/pipeline/detectors/person_detector.py
b/backend/pipeline/detectors/person_detector.py
index 38ec2b3..5b931a4 100644
--- a/backend/pipeline/detectors/person_detector.py
+++ b/backend/pipeline/detectors/person_detector.py
@@ -252,3 +252,50 @@ class PersonDetector:
    windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]

    return windows
+
+    def detections_per_frame(self, video_path: str, start_frame: int, end_frame: int,
+                             frame_skip: int = 3) -> Dict[str, Any]:
+        """Per-frame person detections over `[start_frame, end_frame]` of the ORIGINAL video.
+
+        Used by the fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2). Unlike
+        ``extract_action_windows_from_chunk`` (which detects ITS OWN windows and discards the
+        per-frame boxes), this returns the raw per-frame detections so the fusion features can
+        align them with the shuttle trajectory. It reads the source by **absolute frame index**
+        (seek + sequential read), NOT a re-extracted clip, so the returned frame indices match
+        the WASB trajectory's ``.frame`` exactly ? that alignment is what
+        ``fusion_features.shuttle_player_colocation`` / ``two_sided_motion`` rely on.
+
+        Returns ``{"frames": {frame_idx: [(track_id, bbox), ...], "fps", "frame_width",
+        "frame_height"}``. A fresh window-local ByteTrack tracker is used. ? cv2 frame-seek
+        accuracy varies by codec (it lands near the requested keyframe then decodes forward);
+        treat the indices as ±a few frames on real footage ? fine for window-level features.
+        """
+        self.lazy_load_model()
+        cap = cv2.VideoCapture(video_path)
+        if not cap.isOpened():
+            logger.error(f"Could not open {video_path}")
+            return {"frames": {}, "fps": 30.0, "frame_width": 0.0, "frame_height": 0.0}
+
+        fps = cap.get(cv2.CAP_PROP_FPS)
+        if fps <= 0:
+            fps = 30.0
+        frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
+        frame_height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
+        score_thresh = self.config.get("indexing", {}).get("detector_score_threshold", 0.5)
+        step = max(frame_skip, 1)
+        tracker = self.make_tracker(fps / step)
+
+        start_frame = max(0, int(start_frame))
+        frames: Dict[int, List[Tuple[int, Tuple[float, float, float, float]]]] = {}
+        cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
+        frame_idx = start_frame
+        while frame_idx <= end_frame:
+            ret, frame = cap.read()
+            if not ret:
+                break
+            if (frame_idx - start_frame) % step == 0:
+                frames[frame_idx] = self.detect_and_track(frame, tracker, score_thresh)
+            frame_idx += 1
+        cap.release()
+        return {"frames": frames, "fps": fps,
+                "frame_width": float(frame_width), "frame_height": float(frame_height)}
diff --git a/backend/pipeline/segmenters/__init__.py b/backend/pipeline/segmenters/__init__.py

```

```

index c003d1a..fc5de77 100644
--- a/backend/pipeline/segmenters/__init__.py
+++ b/backend/pipeline/segmenters/__init__.py
@@ -4,6 +4,7 @@ from backend.pipeline.segmenters.gemini import GeminiPlaySegmenter
     from backend.pipeline.segmenters.yolo_hybrid import HybridYoloSegmenter
     from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
     from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
+from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter

__all__ = [
    "segmenter_registry",
@@ -13,4 +14,5 @@ __all__ = [
    "HybridYoloSegmenter",
    "TrackNetHybridSegmenter",
    "WasbHybridSegmenter",
+    "FusionSegmenter",
]

```

13. assistant (2026-06-13T19:09:48.658Z)

Now let me look at the test_config.py and test_trajectory_hybrid_equivalence.py additions.

14. user (2026-06-13T19:09:49.311Z)

```

warning: in the working copy of 'tests/test_trajectory_hybrid_equivalence.py', LF will be
replaced by CRLF the next time Git touches it
diff --git a/tests/test_config.py b/tests/test_config.py
index ec09cd5..5519f06 100644
--- a/tests/test_config.py
+++ b/tests/test_config.py
@@ -25,6 +25,36 @@ def test_load_default_config():
     assert cfg.indexing.ai_max_workers == 5

+def test_fusion_config_defaults_reproduce_control():
+    """The fusion block defaults must be no-ops: wasb substrate, no cues, no gating, no
+mask."""
+    cfg = load_config("/nonexistent/path/12345/config.json")
+    f = cfg.indexing.fusion
+    assert f.substrate == "wasb"
+    assert f.cue_flags == {}           # person cue OFF
+    assert f.min_crossings == 0        # Gate A OFF
+    assert f.min_players == 0          # Gate B OFF
+    assert f.court_polygon is None     # no mask
+    assert f.velocity_threshold == 0.02
+
+def test_fusion_config_file_override():
+    with tempfile.TemporaryDirectory() as td:
+        p = os.path.join(td, "fusion.json")
+        with open(p, "w") as fh:
+            json.dump({"indexing": {"fusion": {
+                "substrate": "tracknet", "min_crossings": 2,
+                "cue_flags": {"person": True}, "court_polygon": [[0.1, 0.1], [0.9, 0.1], [0.5,
+0.9]],
+                }}}), fh)
+        cfg = load_config(p)
+        f = cfg.indexing.fusion
+        assert f.substrate == "tracknet"
+        assert f.min_crossings == 2
+        assert f.cue_flags == {"person": True}
+        assert len(f.court_polygon) == 3
+        # An old config without a fusion block still loads (defaults applied).
+        assert cfg.indexing.wasb.velocity_threshold == 0.02

```



```

+
def test_file_override():
    with tempfile.TemporaryDirectory() as td:
        p = os.path.join(td, "override.json")
diff --git a/tests/test_trajectory_hybrid_equivalence.py
b/tests/test_trajectory_hybrid_equivalence.py
index 495a0cd..dfe58af 100644
--- a/tests/test_trajectory_hybrid_equivalence.py
+++ b/tests/test_trajectory_hybrid_equivalence.py
@@ -100,3 +100,25 @@ def test_base_is_not_registered():
    # The base engine itself must not appear as a selectable segmenter.
    assert not any(isinstance(cls, type) and cls is TrajectoryHybridSegmenter
                    for cls in available.values())
+
+
+@pytest.mark.parametrize("cls,family,detector", TWINS)
+def test_persisted_stats_are_exactly_the_four_motion_keys(cls, family, detector):
+    """The fusion-P2 enrichment seam must leave the twins' persisted stats byte-identical:
+    the base _enrich_candidate_stats is identity, so NO net_crossings / person keys leak in."""
+    db, _res = _run(cls, [{"start_time": 0.5, "end_time": 3.0}])
+    _, kwargs = db.add_candidate_segment.call_args
+    assert set(kwargs["stats"]) == {
+        "peak_velocity", "mean_velocity", "active_frame_count", "track_id_count"}
+
+
+@pytest.mark.parametrize("cls,family,detector", TWINS)
+def test_base_fusion_hooks_are_identity(cls, family, detector):
+    """Directly pin the two fusion seams on the base: stats = the 4 motion keys verbatim,
+    and every window is selected for handoff (no gating)."""
+    seg = cls(MagicMock(), {"indexing": {}})
+    cw = _window()
+    stats = seg._enrich_candidate_stats(cw, [], 30.0, 1280.0, "v.mp4", {})
+    assert stats == {"peak_velocity": 0.4, "mean_velocity": 0.2,
+                    "active_frame_count": 30, "track_id_count": 1}
+    assert seg._select_for_handoff([cw, cw, cw], [stats, stats, stats], {}) == [0, 1, 2]

```

15. assistant (2026-06-13T19:09:51.691Z)

Now let me read the rest of the equivalence test file to understand `\_run`, `\_window`, `TWINS`, and what's mocked, and check the `rally\_gate` API for crossings.

16. user (2026-06-13T19:09:52.474Z)

```

1     """Equivalence contract for the trajectory-hybrid twins.
2
3     tracknet_hybrid and wasb_hybrid were copy-paste twins that drifted (the
4     shot_count parse); both are now thin subclasses of TrajectoryHybridSegmenter.
5     These tests run BOTH through the same scenarios and assert identical P3/P4
6     behavior, so any future divergence has to be introduced deliberately, in the
7     base, for both at once.
8     """
9     from unittest.mock import MagicMock, patch
10
11     import pytest
12
13     from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
14     from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
15     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
16
17     TWINS = [
18         (TrackNetHybridSegmenter, "tracknet", "tracknet_hybrid"),
19         (WasbHybridSegmenter, "wasb", "wasb_hybrid"),
20     ]
21
22

```

```

23     def _window(start=1.0, end=4.0):
24         return {
25             "start": start, "end": end, "active_frame_count": 30,
26             "peak_velocity": 0.4, "mean_velocity": 0.2, "track_id_count": 1,
27             "chunk_index": 0,
28         }
29
30
31     def _runner(windows=None):
32         runner = MagicMock()
33         runner.healthcheck.return_value = (True, "env ok")
34         runner.run_predict.return_value = "out.csv"
35         runner.parse_trajectory_csv.return_value = [MagicMock(visible=True)]
36         runner.trajectory_to_action_windows.return_value = windows if windows is not None
37     else [_window()]
38     return runner
39
40     def _run(cls, provider_segments, db=None):
41         """Run one segmenter class through the standard mocked pipeline."""
42         provider = MagicMock()
43         provider.analyze_video.return_value = (provider_segments, {})
44         db = db or MagicMock()
45         db.add_candidate_segment.return_value = 55
46         db.add_play_segment.return_value = 200
47
48         with patch.object(cls, "_build_runner", return_value=(_runner(), {"weights_path":
49 "w"})), \
50             patch.object(cls, "_probe_fps_width", return_value=(30.0, 1280.0)), \
51             patch("backend.pipeline.segmenters.trajectory_hybrid.get_video_duration",
52 return_value=30.0), \
53             patch("backend.pipeline.segmenters.trajectory_hybrid.extract_video_chunk",
54 return_value=True), \
55             patch("backend.pipeline.segmenters.trajectory_hybrid.provider_registry") as
56 preg, \
57             patch("os.environ.get", return_value="dummy_key"):
58             preg.get.return_value = provider
59             seg = cls(db, {"indexing": {}})
60             return db, seg.process_video("v1", "clip.mp4")
61
62
63 @pytest.mark.parametrize("cls, family, detector", TWINS)
64 def test_detector_identity_flows_into_db_rows(cls, family, detector):
65     db, res = _run(cls, [{"start_time": 0.5, "end_time": 3.0, "shuttle_exchange_count":
66 4}])
67     assert len(res) == 1
68     _, kwargs = db.add_candidate_segment.call_args
69     assert kwargs["detector"] == detector
70     assert kwargs["detection_params"]["detector"] == detector
71     # P4 metadata tags the family + model.
72     assert res[0]["metadata"]["detector"].startswith(f"{family}-hybrid-")
73     db.link_candidate_to_segment.assert_called_once_with(55, 200)
74
75
76 @pytest.mark.parametrize("cls, family, detector", TWINS)
77 @pytest.mark.parametrize("exchange, expected", [
78     (4, 4), # provider-reported count wins
79     ("7", 7), # numeric strings coerce
80     (None, 3), # explicit null -> duration estimate (2.5s/0.8 -> 3 floor)
81     ("garbage", 3), # unparseable -> same estimate
82 ])
83 def test_shot_count_behavior_is_identical(cls, family, detector, exchange, expected):
84     """The historical drift point: wasb_hybrid relied on int(None) raising into
85 the fallback while tracknet_hybrid short-circuited. One implementation now."""
86     seg_dict = {"start_time": 0.5, "end_time": 3.0, "shuttle_exchange_count": exchange}

```

```

82         _, res = _run(cls, [seg_dict])
83         assert res[0]["metadata"]["shot_count"] == expected
84
85
86     @pytest.mark.parametrize("cls,family,detector", TWINS)
87     def test_provider_offsets_applied_identically(cls, family, detector):
88         # Window [1,4] + 2.0s default padding -> clip starts at max(0, -1) = 0.0;
89         # provider-relative 0.5 stays 0.5 absolute here.
90         _, res = _run(cls, [{"start_time": 0.5, "end_time": 3.0}])
91         assert res[0]["start_time"] == 0.5
92         assert res[0]["end_time"] == 3.0
93
94
95     def test_base_is_not_registered():
96         from backend.pipeline.segmenters.base import segmenter_registry
97         available = segmenter_registry.list_available()
98         assert "tracknet_hybrid" in available
99         assert "wasb_hybrid" in available
100        # The base engine itself must not appear as a selectable segmenter.
101        assert not any(isinstance(cls, type) and cls is TrajectoryHybridSegmenter
102                        for cls in available.values())
103
104
105    @pytest.mark.parametrize("cls,family,detector", TWINS)
106    def test_persisted_stats_are_exactly_the_four_motion_keys(cls, family, detector):
107        """The fusion-P2 enrichment seam must leave the twins' persisted stats byte-
108        identical:
109        the base _enrich_candidate_stats is identity, so NO net_crossings / person keys leak
110        in."""
111        db, _res = _run(cls, [{"start_time": 0.5, "end_time": 3.0}])
112        _, kwargs = db.add_candidate_segment.call_args
113        assert set(kwargs["stats"]) == {
114            "peak_velocity", "mean_velocity", "active_frame_count", "track_id_count"}
115
116    @pytest.mark.parametrize("cls,family,detector", TWINS)
117    def test_base_fusion_hooks_are_identity(cls, family, detector):
118        """Directly pin the two fusion seams on the base: stats = the 4 motion keys
119        verbatim,
120        and every window is selected for handoff (no gating)."""
121        seg = cls(MagicMock(), {"indexing": {}})
122        cw = _window()
123        stats = seg._enrich_candidate_stats(cw, [], 30.0, 1280.0, "v.mp4", {})
124        assert stats == {"peak_velocity": 0.4, "mean_velocity": 0.2,
125            "active_frame_count": 30, "track_id_count": 1}
126        assert seg._select_for_handoff([cw, cw, cw], [stats, stats, stats], {}) == [0, 1, 2]
127

```

17. assistant (2026-06-13T19:09:56.328Z)

Now let me check the `rally\_gate.gate\_windows` API to verify how `crossings` works and whether the test's synthetic trajectory actually produces the asserted crossings ? this is central to whether `test\_net\_crossings\_persisted\_and\_person\_off\_by\_default` passes vacuously or genuinely.

18. user (2026-06-13T19:09:56.989Z)

```

1    """Rally-start gate: suppress between-point false fires using shuttle net-crossings.
2
3    WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
4    the opponent for service after losing a point, that single trajectory looks like a
5    rally. The discriminator is structural, not duration-based: a real rally has the
6    shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
7    single service feed crosses ~once.
8
9    This module is camera-agnostic and needs NO new model ? it works on the trajectory

```

```

10 we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
11 axis with the larger shuttle spread) and the net position (median shuttle coord on
12 that axis ? the shuttle clusters at the two ends and crosses the net region often,
13 so the median sits near the net), then counts sign changes of (coord - net) across
14 the visible points inside a candidate window, with a deadband to ignore jitter.
15
16 Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
17 Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
18 windows from trajectory_to_action_windows, or to fold into it later.
19 """
20 from __future__ import annotations
21
22 from dataclasses import dataclass
23 from typing import List, Sequence, Tuple
24
25 Interval = Tuple[float, float]
26
27
28 @dataclass
29 class GatedWindow:
30     start: float
31     end: float
32     crossings: int
33     visible_points: int
34     kept: bool
35
36
37 def _spread(vals: Sequence[float]) -> float:
38     """Robust spread = p95 - p5 (ignores a few outlier detections)."""
39     if len(vals) < 2:
40         return 0.0
41     s = sorted(vals)
42     lo = s[max(0, int(0.05 * (len(s) - 1)))]
43     hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
44     return hi - lo
45
46
47 def _median(vals: Sequence[float]) -> float:
48     if not vals:
49         return 0.0
50     s = sorted(vals)
51     n = len(s)
52     return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])
53
54
55 def estimate_play_axis(points) -> Tuple[str, float, float]:
56     """Return (axis 'x'/'y', net_value, span) from visible points.
57
58     Play axis = the image axis along which the shuttle travels most (larger spread).
59     For a baseline camera that's Y (players top/bottom); for a side camera, X.
60     """
61     xs = [p.x for p in points if p.visible]
62     ys = [p.y for p in points if p.visible]
63     sx, sy = _spread(xs), _spread(ys)
64     if sx >= sy:
65         return "x", _median(xs), sx
66     return "y", _median(ys), sy
67
68
69 def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
70     """Count sign changes of (coord - net) across visible points, ignoring the
71     deadband zone near the net (so jitter at the net doesn't inflate the count)."""
72     crossings = 0
73     last_side = None
74     for p in window_points:

```

```

75         if not p.visible:
76             continue
77         v = (p.x if axis == "x" else p.y) - net
78         if abs(v) < deadband:
79             continue
80         side = v > 0
81         if last_side is not None and side != last_side:
82             crossings += 1
83         last_side = side
84     return crossings
85
86
87     def _points_in(points, fps: float, start: float, end: float):
88         return [p for p in points if start * fps <= p.frame <= end * fps]
89
90
91     def gate_windows(points, windows: List[Interval], fps: float,
92                     min_crossings: int = 2, deadband_frac: float = 0.06) ->
List[GatedWindow]:
93         """Annotate each (start,end) window with its shuttle net-crossing count and a
94         keep decision (crossings >= min_crossings). Deadband is a fraction of the play
95         axis span. Net/axis are estimated once over the whole trajectory (stable)."""
96         axis, net, span = estimate_play_axis(points)
97         deadband = deadband_frac * span
98         out: List[GatedWindow] = []
99         for (s, e) in windows:
100             wp = _points_in(points, fps, s, e)
101             c = count_net_crossings(wp, axis, net, deadband)
102             vis = sum(1 for p in wp if p.visible)
103             out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
104                                   kept=c >= min_crossings))
105         return out
106
107
108     def apply_gate(points, windows: List[Interval], fps: float,
109                   min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
110         """Convenience: return only the windows that pass the crossing gate."""
111         return [(g.start, g.end) for g in
112                 gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
113

```

19. assistant (2026-06-13T19:10:01.935Z)

Now I understand the rally_gate math. Let me verify a critical concern in the fusion test: the synthetic trajectory. The `estimate\_play\_axis` runs over ALL points (window A: x=500, y flips 100/900; window B: x=500, y=500). So xs are all 500 ? spread 0. ys range 100-900 ? spread ~800. So axis='y', net=median(ys). Let me verify whether the test's net_crossings assertions for window B (held, y=500) actually hold given the deadband and that the net is the median.

Let me run the actual tests to see if they pass and check coverage, plus inspect a few more things.

20. user (2026-06-13T19:10:11.582Z)

```

..... [100%]
42 passed in 6.12s

```

21. assistant (2026-06-13T19:10:14.138Z)

All pass. Now let me dig into the key correctness questions. First, the net-crossings math for the synthetic trajectory to confirm the assertions aren't vacuous.

22. user (2026-06-13T19:10:18.009Z)

```
axis y net 500.0 span 800.0 deadband 48.0
A crossings 89 visible 90
B crossings 0 visible 60
```

23. assistant (2026-06-13T19:10:23.748Z)

Window A genuinely produces 89 crossings, window B genuinely 0 (held at net level, inside deadband). So ``test_net_crossings_persisted_and_person_off_by_default`` is NOT vacuous ? good. Now let me examine the most suspicious test: ``test_person_cue_adds_features_when_enabled``. The patch target is ``backend.pipeline.detectors.person_detector.PersonDetector``, but ``_person_features`` does a lazy import ``from backend.pipeline.detectors.person_detector import PersonDetector``. Let me verify the patch actually intercepts and whether ``detections_per_frame`` is genuinely exercised vs. the mock returning a canned dict regardless.

Let me check whether the person-cue test actually verifies the feature `*values*` or just key presence, and whether the ``_person_features`` real logic (frame range, shuttle alignment, polygon/net wiring) is exercised at all.

24. user (2026-06-13T19:10:30.219Z)

```
WIN_A feats: {'players_in_court': 1.0, 'two_sided_motion': 0.0, 'mean_player_motion':
0.0019999999999999983, 'in_court_ratio': 0.0, 'shuttle_player_colocation': 0.0}
WIN_B feats: {'players_in_court': 1.0, 'two_sided_motion': 0.0, 'mean_player_motion':
0.0019999999999999983, 'in_court_ratio': 0.0, 'shuttle_player_colocation': 0.0}
```

25. assistant (2026-06-13T19:10:37.828Z)

Important finding: the person-cue test feeds the `**same**` mocked ``detections_per_frame`` return for BOTH windows (frames 30, 31), so both windows get identical person features ? and crucially ``shuttle_player_colocation`` is 0.0 for both because the mocked detection frames (30, 31) only align with window A's shuttle, but the shuttle at frame 30/31 is at y=100/900 (x=500), far from box (10,10,30,40). The test only asserts key presence, never values. So the wiring inside ``_person_features`` (frame-range slicing ``start_f<=p.frame<=end_f``, polygon parsing, net tuple construction, ``det["frame_width"]` or `frame_width`` fallback) is `**not verified for correctness**` ? only that keys appear.

Now let me check a subtle bug concern: the patch target. The test patches ``backend.pipeline.detectors.person_detector.PersonDetector`` and ``_person_features`` imports it lazily from that module ? so patch works. But let me confirm whether ``PersonDetector(self.config)`` is even constructed correctly (the test passes ``{"indexing": {"fusion": {...}}}`` as config). Also note ``frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)`` ? ``self.config`` is a plain dict in tests. Let me verify the real config object behavior.

26. user (2026-06-13T19:10:39.951Z)

2c241a8 refactor: collapse the hybrid-segmenter twins into a shared trajectory base

```
def _select_for_handoff(self, windows: List[Dict[str, Any]],
                        window_stats: List[Dict[str, Any]],
                        idx_cfg: Dict[str, Any]) -> List[int]:
    """Indices of the candidate windows that proceed to the AI handoff (and thus may
    become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
    ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
    Persisted candidates are NOT affected ? they remain the full superset for offline
    re-scoring."""
    return list(range(len(windows)))

def process_video(self, video_id: str, video_path: str, # type: ignore[override]
                  player_pool: Optional[List[str]] = None,
                  max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
    # override-ignore: the ABC declares the Result contract (Success|Failure)
    # but every live segmenter still returns a bare list ? the documented
```

```

# deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
label = self.display_label
# --- Sport profile + AI provider wiring (identical across segmenters) ---
sport_name = self.config.get("sport", "badminton")
try:
    sport_profile = sport_registry.get(sport_name)
except KeyError as e:
    logger.error(str(e))
    return []

idx_cfg = self.config.get("indexing", {})
provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider", "gemini"))
model_name = self.config.get("ai_model", idx_cfg.get("ai_model", "gemini-2.5-flash"))

api_key_env_var = f"{provider_name.upper()}_API_KEY"
api_key = os.environ.get(api_key_env_var)
if not api_key:
    logger.error(
        f"{api_key_env_var} environment variable is not set! "
        f"To run the {provider_name} handoff, set your API key. "
        f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
    )
    return []
try:
    provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
except KeyError as e:
    logger.error(str(e))
    return []

video_duration = get_video_duration(video_path)
if video_duration <= 0:
    logger.error("Invalid video duration.")
    return []
fps, frame_width = self._probe_fps_width(video_path)

# --- Runner config + windowing thresholds ---
runner, runner_params = self._build_runner(idx_cfg)

velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
ai_max_workers = idx_cfg.get("ai_max_workers", 5)
log_candidates = idx_cfg.get("log_yolo_candidates", True)
store_candidates = idx_cfg.get("store_yolo_candidates", True)

detection_params = {
    "detector": self.name,
    "velocity_thresh": velocity_thresh,
    "merge_gap": merge_gap,
    "min_action_window_duration": min_window_duration,
    **runner_params,
}
}

# --- Phase 1: env healthcheck (fail fast with a clear message) ---
ok, msg = runner.healthcheck()
if not ok:
    logger.error("%s WSL environment not ready: %s", label, msg)
    return []
logger.info("%s WSL env OK: %s", label, msg)

# --- Phase 2: trajectory -> candidate rally windows ---
# Trajectory detectors process the whole video in one pass (they carry
# their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.

```

```

t_start = time.time()
out_dir = os.path.join("output", self.family)
csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
if not csv_path:
    logger.error("%s produced no trajectory; aborting.", label)
    return []

points = runner.parse_trajectory_csv(csv_path)
logger.info("Parsed %d trajectory points (%d visible).",
            len(points), sum(1 for p in points if p.visible))

all_candidate_windows = runner.trajectory_to_action_windows(
    points, fps=fps, frame_width=frame_width, chunk_start=0.0,
    velocity_thresh=velocity_thresh, merge_gap=merge_gap,
    min_window_duration=min_window_duration, chunk_index=0,
)
logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
            label, time.time() - t_start, len(all_candidate_windows))

if log_candidates:
    for cw in all_candidate_windows:
        logger.info(f"[{label} rally] {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
                    f"({cw['end'] - cw['start']:.1f}s) | "
                    f"peak_v={cw['peak_velocity']:.3f} "
                    f"mean_v={cw['mean_velocity']:.3f}")

# Per-window stats via the enrichment hook ? base = the 4 motion keys; the fusion

```

27. assistant (2026-06-13T19:10:45.129Z)

Now I have a clear view. Let me check one more crucial thing: the `\_select\_for\_handoff` Gate B path (`min\_players`) ? is it tested at all? And the `substrate == "tracknet"` branch in `\_build\_runner`. And whether `velocity\_threshold` from the fusion config is actually read by the engine (the config defines `fusion.velocity\_threshold` but the engine reads `idx\_cfg.get(self.family, ...)` where `self.family = "fusion"` ? so it reads `idx\_cfg["fusion"]["velocity\_threshold"]`, which matches). Let me verify the family-keyed velocity read and check whether `min\_players` Gate B is exercised.

28. user (2026-06-13T19:10:56.362Z)

```

family: fusion
Gate B keep: [0]
Gate B (person off) keep: [0, 1]

```

29. assistant (2026-06-13T19:11:00.094Z)

Gate B works but is ****not tested anywhere**** in the suite. Now let me check the tracknet substrate branch and whether `test\_fusion\_config\_file\_override` actually validates the tracknet `weights\_path` requirement (TrackNetConfig may require a weights path). Let me also verify the `point\_in\_polygon` edge-inside assertion at line 47 and the `\_shuttle\_in\_box` pad behavior coverage.

30. user (2026-06-13T19:11:05.585Z)

```

pad-adjacent inside? True
beyond pad? False
---tracknet branch---
True

```

31. assistant (2026-06-13T19:11:09.140Z)

Now let me confirm the `two\_sided\_motion` court_polygon parameter is never exercised with a non-None polygon (the polygon filter inside two_sided_motion), and check whether `person\_in\_court`

with unusable dims line-47 assertion is testing real behavior or a misleading comment.

32. user (2026-06-13T19:11:15.970Z)

```
foot at 0 dims: (0.0, 0.0)
point_in_polygon((0,0),LEFT_HALF): True
person_in_court 0 dims: True
two_sided FULL poly: 1.0
two_sided NEAR_ONLY poly: 0.0
```

33. assistant (2026-06-13T19:11:25.001Z)

The line-47 assertion's comment "(0,0) is on left half edge-inside" is correct behavior but the test is asserting a degenerate/accidental property (foot collapses to origin on bad dims, and origin happens to be polygon-inside). That's testing an implementation artifact, not a meaningful behavior ? though it does pin current behavior.

Now let me check one genuinely important untested behavior: does the fusion segmenter actually pass `fusion.velocity\_threshold` and `fusion.substrate` through correctly in the full `process\_video` path, and does the `tracknet` substrate branch get covered? Also let me verify whether `test\_persisted\_candidates\_are\_superset\_of\_substrate\_windows` actually checks the gated case keeps the superset (it uses default config = no gating, so it doesn't prove the "superset under gating" claim). The Gate A test `test\_gate\_a\_drops\_low\_crossing\_window\_from\_handoff\_only` does check `len(\_stats\_calls(db)) == 2` under gating ? good. Let me verify the key claim about colocation alignment that the docstring stresses (frame-index alignment) is never tested through the segmenter.

Let me check the `detections\_per\_frame` method's actual logic for a no-GPU testability gap ? it uses cv2 but the frame-skip/seek/loop logic is pure given a mock cap.

34. user (2026-06-13T19:11:27.584Z)

```
tests/test_fusion_hybrid.py:100:    fake_pd.detections_per_frame.return_value = {
tests/test_fusion_hybrid.py:109:    fake_pd.detections_per_frame.assert_called()                # the
person detector WAS used
Binary file tests/__pycache__/test_fusion_hybrid.cpython-313-pytest-9.0.3.pyc matches
--- check if person_detector detections_per_frame logic is tested ---
tests/test_person_detector.py:39:    pd.model = MagicMock()                #
short-circuits lazy_load_model
tests/test_person_detector.py:40:    pd.make_tracker = MagicMock(return_value=MagicMock())
tests/test_person_detector.py:41:    pd.detect_and_track =
MagicMock(side_effect=_make_detections())
tests/test_person_detector.py:60:    def
test_detect_and_track_filters_persons_and_returns_track_ids():
tests/test_person_detector.py:61:    """detect_and_track keeps only confident PERSON detections,
feeds them to the
tests/test_person_detector.py:95:    result = pd.detect_and_track("frame", tracker,
score_thresh=0.5)
tests/test_yolo_hybrid.py:32:    """Per-frame return values for a mocked detect_and_track: one
steadily-moving
tests/test_yolo_hybrid.py:46:    segmenter.person.model = MagicMock()                # short-
circuits lazy_load_model
tests/test_yolo_hybrid.py:47:    segmenter.person.make_tracker =
MagicMock(return_value=MagicMock())
tests/test_yolo_hybrid.py:48:    segmenter.person.detect_and_track =
MagicMock(side_effect=_make_detections(num_frames, step))
```

35. assistant (2026-06-13T19:11:34.910Z)

This confirms a significant gap: `PersonDetector.detections\_per\_frame` (the NEW method, the only real GPU-adjacent code that has testable pure logic) is **never tested** ? the fusion test mocks the whole method out. Yet `test\_person\_detector.py` already demonstrates the pattern (mock `model`/`make\_tracker`/`detect\_and\_track`, drive `cv2.VideoCapture` with a fake) for testing exactly this kind of loop in a no-GPU container. The seek/skip/frame-index loop in

`detections\_per\_frame` (lines 286-298 of person_detector.py) is the kind of off-by-one-prone logic that should be covered here.

Let me confirm the existing test_person_detector.py pattern to be precise about how `cv2.VideoCapture` is mocked, so my recommendation is concrete.

36. user (2026-06-13T19:11:35.474Z)

```
1      """Unit tests for the person-detection rally cue
(backend.pipeline.detectors.person_detector).
2
3      Pure/offline ? the torchvision model and supervision tracker are mocked, so these run
4      without GPU/torch weights. They pin the COCO person==1 filter (the YOLO-was-0 gotcha)
and
5      the motion-window extraction that feeds the candidate `stats` + the learned fusion
features.
6      """
7      from unittest.mock import MagicMock, patch
8
9      from backend.pipeline.detectors.person_detector import PersonDetector, _PERSON_CLASS_ID
10
11
12      def _make_cap_mock(num_frames=300):
13          """A cv2.VideoCapture mock yielding `num_frames` readable frames at 30fps."""
14          import cv2
15          cap_mock = MagicMock()
16          cap_mock.isOpened.return_value = True
17
18          def mock_get_prop(prop):
19              if prop == cv2.CAP_PROP_FPS:
20                  return 30.0
21              if prop == cv2.CAP_PROP_FRAME_WIDTH:
22                  return 1000.0
23              return 0.0
24
25          cap_mock.get.side_effect = mock_get_prop
26          cap_mock.read.side_effect = [(True, "frame")] * num_frames + [(False, None)]
27          return cap_mock
28
29
30      def _make_detections(num_frames=300, step=5, track_id=1):
31          """One steadily-moving person per frame (box advances `step` px/frame)."""
32          return [(track_id, (float(i * step), 0.0, float(i * step + 10), 10.0))]
33              for i in range(num_frames)]
34
35
36      @patch("backend.pipeline.detectors.person_detector.cv2.VideoCapture")
37      def test_extract_action_windows_returns_enriched_dicts(mock_cap):
38          pd = PersonDetector({"indexing": {"min_action_window_duration": 1.0}})
39          pd.model = MagicMock() # short-circuits
lazy_load_model
40          pd.make_tracker = MagicMock(return_value=MagicMock())
41          pd.detect_and_track = MagicMock(side_effect=_make_detections())
42          mock_cap.return_value = _make_cap_mock()
43
44          windows = pd.extract_action_windows_from_chunk(
45              "chunk.mp4", chunk_start=5.0, velocity_thresh=0.05, merge_gap=2.0,
46              frame_skip=1, chunk_index=2,
47          )
48
49          assert len(windows) >= 1
50          w = windows[0]
51          assert set(w) >= {"start", "end", "active_frame_count", "peak_velocity",
52                          "mean_velocity", "track_id_count", "chunk_index"}
53          assert w["chunk_index"] == 2
```

```

54         assert w["start"] >= 5.0                                # full-video coordinates
55         assert w["active_frame_count"] > 0
56         assert w["peak_velocity"] >= w["mean_velocity"] > 0
57         assert w["track_id_count"] == 1
58
59
60     def test_detect_and_track_filters_persons_and_returns_track_ids():

```

37. assistant (2026-06-13T19:11:39.495Z)

This confirms the existing harness pattern is directly reusable for `detections\_per\_frame`. I now have everything I need. Let me verify one last thing ? whether the `frame\_skip` modulo and the `det["frame\_height"] or 0.0` ? `fh=0.0` fallback would silently zero out features (a real latent bug worth noting that no test catches).

38. user (2026-06-13T19:11:44.368Z)

```

fh=0 feats: {'players_in_court': 1.0, 'two_sided_motion': 0.0, 'mean_player_motion': 0.0,
'in_court_ratio': 0.0, 'shuttle_player_colocation': 0.0}
fh=1000 feats: {'players_in_court': 1.0, 'two_sided_motion': 0.0, 'mean_player_motion':
0.001999999999999983, 'in_court_ratio': 0.0, 'shuttle_player_colocation': 1.0}

```

39. assistant (2026-06-13T19:11:48.780Z)

This confirms a real latent issue: when `frame\_height` is 0 (note line 99-100 of fusion_hybrid.py: `fh = det["frame\_height"] or 0.0` ? there's a fallback for `frame\_width` to the substrate's `frame\_width`, but `frame\_height` has NO fallback), the person features silently collapse to near-zero, and no test catches this asymmetry. Note `frame\_width` falls back to `frame\_width` (the substrate value) but `frame\_height` falls back to `0.0`.

I have a comprehensive picture now. Let me compile the findings.

40. user (2026-06-13T19:12:42.500Z)

Structured output provided successfully

41. assistant (2026-06-13T19:13:01.284Z)

I've completed the audit. All 42 tests pass, and the synthetic-trajectory net-crossing assertions are genuinely non-vacuous (window A produces 89 crossings, window B 0). Here is the summary of findings, ranked by severity.

High severity

1. ****`PersonDetector.detections\_per\_frame` has zero test coverage**** (`backend/pipeline/detectors/person_detector.py:255-301`). This is the only new non-pure code, and its seek/stride/boundary loop is the off-by-one-prone part. The fusion test mocks the entire method out (``assert_called()`` only). This is ****not**** a GPU gap ? `tests/test_person_detector.py` already proves the exact CPU harness (mock ``cv2.VideoCapture`` + ``detect_and_track`` side_effect) needed to drive it. The frame-index alignment that the colocation/two-sided features depend on is asserted nowhere.

2. ****`frame\_height` has no fallback (asymmetric with `frame\_width`)**** (`fusion_hybrid.py:99-100`). ``fw = det["frame_width"] or frame_width`` recovers from a 0 read, but ``fh = det["frame_height"] or 0.0`` does not. With ``fh=0`` (flaky per-codec, as the method's own docstring warns) every person feature silently collapses to ~0 ? every gated window looks like dead time. Confirmed: ``mean_player_motion` 0.002?0.0`, ``shuttle_player_colocation` 1.0?0.0`. The person-cue test hard-codes ``frame_height=1000.0``, dodging it.

3. ****Gate B (``min_players``) is completely untested**** (`fusion_hybrid.py:124`). Gate A is covered; Gate B and its ``person_on`` guard are not. A regression applying ``min_players`` regardless of the person flag would stay green.

Medium severity

4. **``test\_person\_cue\_adds\_features\_when\_enabled`` asserts only key-presence****
(`test\_fusion\_hybrid.py:98-109`). Same canned detections for both windows;
`shuttle\_player\_colocation` computes to 0.0 for both. None of the real `\_person\_features` wiring
(frame-range slice, polygon parse, net tuple, fw/fh fallback) is validated.
5. **Pad-adjacency and non-None court_polygon branches untested**** (`test\_fusion\_features.py`).
The whole point of `shuttle\_player\_colocation` is the `pad\_frac` tolerance, but test points are
dead-centre-inside or far-outside ? the discriminating boundary is never pinned.
`two\_sided\_motion`'s polygon filter is only ever called with `None`.
6. **The "superset" test doesn't prove the superset-under-gating claim****
(`test\_fusion\_hybrid.py:79-83`). It runs with default (no-gating) config where
persistence==handoff trivially. The actual invariant is proven by the Gate A test instead.

Low severity

7. **The `tracknet` substrate branch of `\_build\_runner` is never exercised****
(`fusion\_hybrid.py:58-68`) ? every test patches `\_build\_runner` out. The routing +
`runner\_params` dict are pure and unit-testable.
8. **``person\_in\_court(in\_box, 0, 0, LEFT\_HALF) is True`` pins an implementation artifact****
(`test\_fusion\_features.py:46-47`) ? it locks in "bad dims ? counted at origin ? in-court," a
dubious behavior dressed up as graceful degradation (and it compounds finding #2).

Relevant files: `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/person\_detector.py`, `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/segmenters/fusion\_hybrid.py`,
`C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/fusion\_features.py`, `C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test\_fusion\_hybrid.py`, `C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test\_fusion\_features.py`, `C:/Users/avidu/Projects/badminton-highlight-indexer/tests/test\_person\_detector.py` (the reusable harness).